

Semi-Autonomous Wildfire Monitoring and Control System Framework

Gene Foxwell

Woolf University / Udacity School of Artificial Intelligence

Industry Integrated AI-Synthesis Project

Overview and Industry Context

In recent decades, climate change has increased wildfire vulnerability in forests across many regions, including Canada (Wang 2025). As temperatures continue to rise, this threat is unlikely to decline soon. Wildfire response requires substantial resources and creates serious risks to human life (Belval 2025). This paper proposes a framework for a semi-autonomous wildfire monitoring and control system. It draws on techniques from previous projects, explaining how they are integrated and why they were selected. The paper then presents the system design, examines ethical considerations, reviews framework testing results, and discusses the project's relevance for organizations seeking AI professionals.

Integration Rationale

Three primary AI techniques were used in the design of this project:

1. **Semantic Segmentation:** We wanted a fast way to provide the system with predictions of where the fire would go next. Assuming we had either a high-altitude aerial view, or at a periodic satellite view, we decided that building a “risk” map of the area would be a good approach. A risk map in our case is a simple binary classification problem for each cell in our simulation – safe or not safe. Semantic Segmentation, with its ability to classify images on a pixel-by-pixel basis, seemed to be right tool for the job. With this in mind, we reused the UNet (Ronneberger 2015) built for Project 4, with a few modifications that will be covered in the next section.

2. **Support Vector Machine:** Aviation injuries are the primary dangers fire fighters face when fighting wildfires (Centers for Disease Control and Prevention 2014). To this end, we imagine this system would work best with UAVs. NASA already has an example of this (Merlin 2009). While these UAVs will take commands from a central command post (much as NASA designed in their paper), we still need them to work independently. For this simulation, we trained a Support Vector Machine to control the navigation of the UAVs when they don't have a predetermined waypoint. SVMs are fast and lightweight, making them ideal for this task. We used SVMs in a planetary science context for Project 3.
3. **Agentic AI:** A Semi-Autonomous command center was needed to provide commands to the UAVs. We chose to build this using an Agentic AI approach. The approach exploits the reasoning and generative abilities of modern Large Language Models (in our case Claude Sonnet 4.5 was used) to prioritize fire fighting activities and generate waypoints for the UAVs. GraphRAG (Han 2025) was introduced on top of this to allow the system to store long term information about its goals and enhance reasoning abilities. This was based on Project 6, where similar architecture was developed for generating Interactive Fiction environments. We consider the system to be semi-autonomous as well allow the user to pause the system and query/provide new priorities at any time while the simulation is active.

In addition to the above, we originally intended to use a Decision Transformer (Chen 2021) as outlined in Project 5 on Generative AI. However, Decision Transformers require a significant number of expert training examples to make work, as overlapping in many

ways with the capabilities of the Agentic AI system. For this reason, we have chosen not to apply that technique on this project.

System Design and Technical Decisions

We split the system design into three distinct modules, as described below. A visual overview of the system design can be found in Figure 5.

Environmental Module

This module was responsible for simulating the environment as well as generating “risk predictions” based on the environmental inputs.

Our environment was randomly generated using Perlin Noise, adapting a technique from (Kamble 2025). We effectively assigned environmental features like grasslands, trees, water, housing, and urban areas to numerical ranges – converting the generated noise into terrain. Figure 1 shows an example of the terrain this generates.

A similar approach was used to generate a static model for wind, computing wind direction based on the output from a separate Perlin Noise iteration. To keep the simulation tractable, we assumed wind to be static once generated.

We simulated the Wildfire itself using a Stochastic Cellular Automata approach adapted from (Ghosh 2024) and (Ramadan 2024). We begin each simulation by igniting a single “cell” in the automata and letting the fire play out. We used equation 1 to determine the probability of fire spreading from one cell to another. An example run through of the system can be seen in Figure 2.

Equation 1:

$$p_{ignite} = [1 + \exp(-k * burningtrees - windalignment - burnthreshold)]^{-1}$$

VARIABLE	INTERPRETATION
K	A scaling parameter.
BURNINGTREES	The number of burning cells adjacent to this cell.
WINDALIGNMENT	The dot product of the average wind direction and average fire direction about this cell.
BURNTHRESHOLD	A constant determining the minimum probability we need to “ignite” a cell.

Each burnable cell has a finite amount of fuel associated with it. This fuel is decremented at the end of every step of the simulation. When a cell runs out of fuel, it is converted to ash.

Finally, one element we felt was missing from previous simulations was the presence of humans in the forested area. Given that humans do sometimes head out into the forest for various work and recreational reasons, we added human agents to the simulation. These human agents are purely stochastic and are considered “casualties” when they encounter fire.

Risk Predictor (Sub-Module of Environment Module)

We trained a modified version of a UNet to perform risk prediction. Our primary modification was to alter the bottleneck to use multiple 3x3 convolutional layers instead of a single 1x1 convolution. We did this as early version showed a tendency to copy the original locations of the

fire and ignore future locations – it was hoped this this additional complexity would allow it to predict risk status and not simply classify fire. Figure 3 shows a schematic diagram of our UNet.

We created a synthetic dataset using output from our simulation. We recorded a random starting point in the simulation, ran the simulation for 20 steps, then recorded the output. The areas in the final image that were still burning or destroyed (but not destroyed in the original image) were used to create a mask. Red Pixels were considered “at risk” areas, while green was considered safe. An example of the results after training can be seen in Figure 4.

Our model was trained using the following parameters:

PARAMETER	VALUE
EPOCHS	10
LEARNING RATE	0.001
OPTIMIZER	AdamW
LEARNING RATE SCHEDULER	Mode=minimization, factor=0.5, patience=3

We used a hybrid loss function, as shown in Equation 2

Equation 2:

$$loss = 0.5 * BCELossWithLogitsLoss + 0.5 * DiceLoss$$

The BCEWithLogitsLoss guides the system towards handling binary classification on a per-pixel basis, while the DiceLoss helps ensure that the overall shape of the output is preserved. Two loss functions were chosen to help mitigate cases where very few pixels would be

considered “at risk”. Early stopping was used to avoid overfitting. The resulting loss curves can be seen in Figure 7.

We evaluated the model using Mean IoU and Dice Score:

CLASS & METRIC	SCORE
SAFE – MIOU	0.9422
SAFE – DICE	0.9698
RISKY – MIOU	0.6205
RISKY - DICE	0.7564

UAV Module

We extrapolate from (Merlin 2009) the potential existence of three classes of UAV: Recon (a direct analogue to the UAV in Merlin 2009), Extinguishers equipped to put out fires, and Search and Rescue UAVs that can pick up humans from unsafe areas.

Our UAVs used a hybrid approach of combining deterministic behavior with behavior guided by a machine learning model (an SVM in our instance). When given a waypoint, UAVs proceed directly to the waypoint. If a UAVs primary action can be performed in the cell it is in, it does so automatically. When the UAV does not have a waypoint, it uses the SVM to determine its navigational commands. This approach is akin to subsumption where one behavioral layer

overrides the ones beneath it. Connell 1990 provides an in-depth example of this approach.

Figure 8 shows an overview of how these behaviors override each other.

We trained the SVM in two stages:

First, we used Monte-Carlo + Q-Learning (Watkins 1992) to iteratively learn a greedy policy using a Q-Value function. This Q-Value function was estimated using a Random Forest (SVR was rather slow). Once we had an estimate of the Q-Value function, we used that to train a Support Vector Machine to learn to the best action based on the inputs.

Each UAV class takes in as input the cells in the UAVs detection radius. Detection radius varies by UAV. We have three distinct reward functions – one for each class of UAV. Reward function design was inspired by (Yu 2025) and (Julian 2018). Reward function design approach was to divide rewards into Extrinsic or Goal Based parameters and Intrinsic Behavioral Parameters. Reward functions for each UAV class are listed below:

Recon:

$$reward = -0.5 - dist(agent_{pos}, agent_{waypoint}) + burningcellcount + 2 * humancount$$

Extinguisher:

$$reward = -0.5 - dist(agent_{pos}, agent_{waypoint}) + 25 * extinguishedfire \\ - remainingfire$$

Search and Rescue:

$$reward = -0.5 - dist(agent_{pos}, agent_{waypoint}) + 50 * rescuedhumancount + 5 \\ * detectedhumancount$$

Constants were chosen via intuition with some minor tuning while performing end-to-end testing. Unfortunately, optimizing these constants would require further testing on the full system which would prohibitively expensive.

We performed 10 training rounds. For each training round we collected 100 episodes per agent. After training, we measured the mean reward and standard deviation for each agent, which is listed below:

UAV	MEAN REWARD	STANDARD DEVIATION
RECON	-5.99	6.05
EXTINGUISH	-6.9	11.05
RESCUE	-6.02	4.36

These results might improve with the application of techniques like Experience Replay, but these have been left for a future project.

In addition to their own behaviors, the UAVs have functionality intended to facilitate interaction with the Command Module. UAVs keep a record of all observations / positions / waypoint status for each step. This data is sent to the Command Module by the simulation after an appropriate number of steps have passed. UAVs also accept commands from the Command Module (via the simulation) setting new waypoints / sending back to homebase for refueling.

Command Module

We used Agentic Workflow as the basis for our Command Module. We modelled the Command Module as a multi-agent architecture. Figure 6 provides a schematic diagram of the workflow.

Each Agent other than the Command Centre Agent (which is more of an interface between the simulation and the Agentic Workflow) keeps a memory of previous interactions which are inserted as part of the prompt for each turn. The memory is implemented as a simple deque with max length of 5. In addition, to avoid exceeding the input token limit on our model, we truncate the end of the input. Historical data is fed to the agent in reverse order, so the newest inputs are first and the oldest are last. This way any data that is truncated is related to older interactions rather than newer ones.

The system tracks long term goals and priorities using a Knowledge Graph, an approach adapted from (Yang 2025). We assume three long term goals in our model: Search and Rescue, Housing Protection, and Fire Fighting. We provide the system with predefined ontology for tracking subgoals of these long-term goals (as seen in Figure 6).

Our Knowledge Graph is implemented using MGraphDB¹, a python library that allows us to create an in-memory Graph Database. We created a wrapper around this that exposes tools for the Agents to add/remove/query various entities and relationships from the graph as needed.

Our workflow consisted of the following agents:

AGENT	PURPOSE	KNOWLEDGE GRAPH ACCESS
Command Centre Agent	Orchestration. The command center agent takes in the	

¹ [mgraph-db · PyPI](#)

	inputs from the simulation and outputs the final response from the Workflow.	
Strategy Agent	This agent is responsible for developing and tracking global strategy – which areas and activities should be focused on, which can be deprioritized. It generates a collection of Activities each with a priority attached to them.	Has full read/write access to the underlying Knowledge Graph.
Dispatch Agent	The Dispatch Agent translates the activities and priority list generated by the Strategy Agent into waypoints to send the UAVs.	Has Read only access to the underlying Knowledge Graph
Assistant Agent	The Assistant Agent responds to user queries and update the Long Term Goals in the Knowledge Graph as needed.	Has full read/write access.

System prompts for each agent have been included in the Appendix.

An outline of how this was integrated with the rest of the simulation is as follows:

1. On the first step and every 40-simulation step afterwards², the simulation calls the Command Center Agent. The simulation provides the recent history of each UAV, a screenshot of the current area, as well as the image mask generated by Fire Prediction UNet model.
2. On the first invocation of the Command Center Agent, and every 5 iterations afterwards, the Strategy Agent is invoked to update the long-term priorities. The Strategy Agent takes in all the input given to the Command Center Agent, updates the Knowledge Graph, and produces a new set of priorities.
3. The most recently collected set of priorities, as well as the input, are sent to the Dispatcher Agent. The dispatch agent reads any additional information it needs from the Knowledge Graph and produces a new waypoint for each UAV under its control. A reason for each command provided.
4. These commands are passed on to the UAVs in the simulation, who update their state. The simulation then proceeds as normal.

At any given time, the Operator can interrupt the simulation while it is running to query the current state of the Knowledge Graph or provide new priorities. These queries are routed to the Assistant Agent, which is responsible for providing answers to user questions and making updates to the Knowledge Graph on their behalf.

² Running the Agents every 40 steps keep the simulation run time down to a manageable amount of time, as well as saving on token usage.

The simulation ends either after 750 steps, or when all fire has been extinguished. At the end of the simulation, a video is saved for further review.

Qualitative and Quantitative results will be covered in the Evaluation and Reflection section.

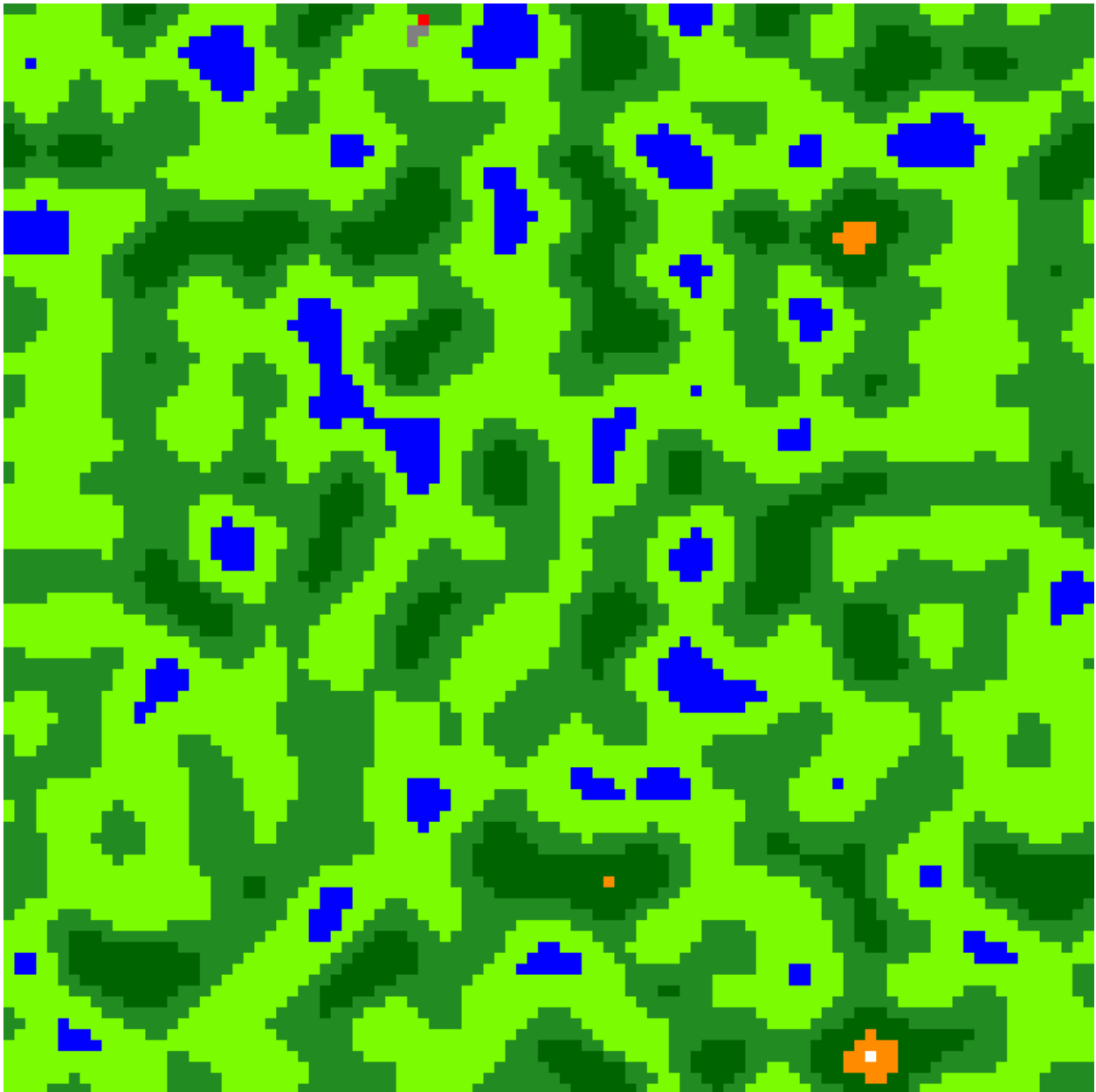


Figure 1 Perlin Noise Generated Terrain

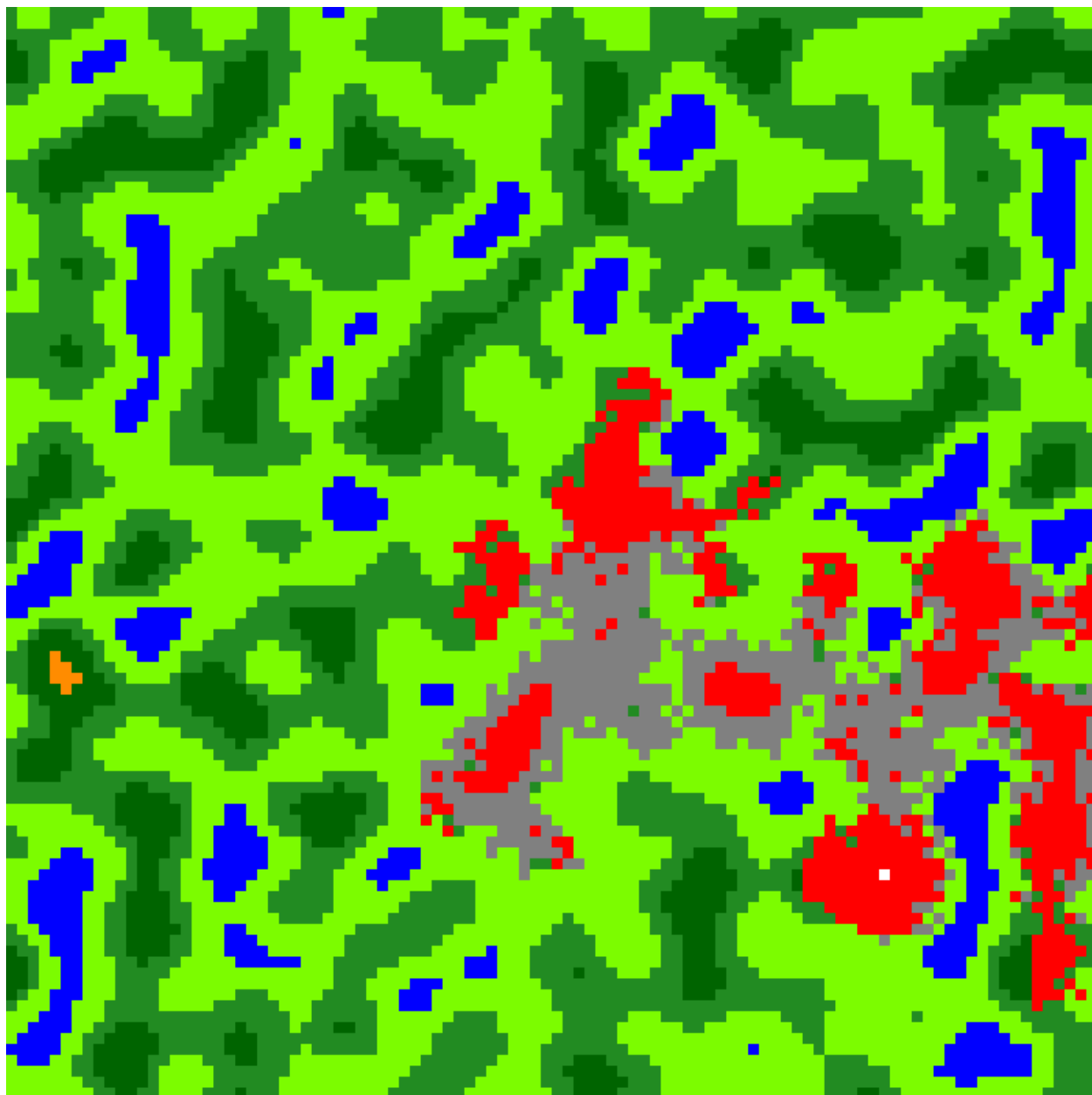


Figure 2 Wildfire CA in action

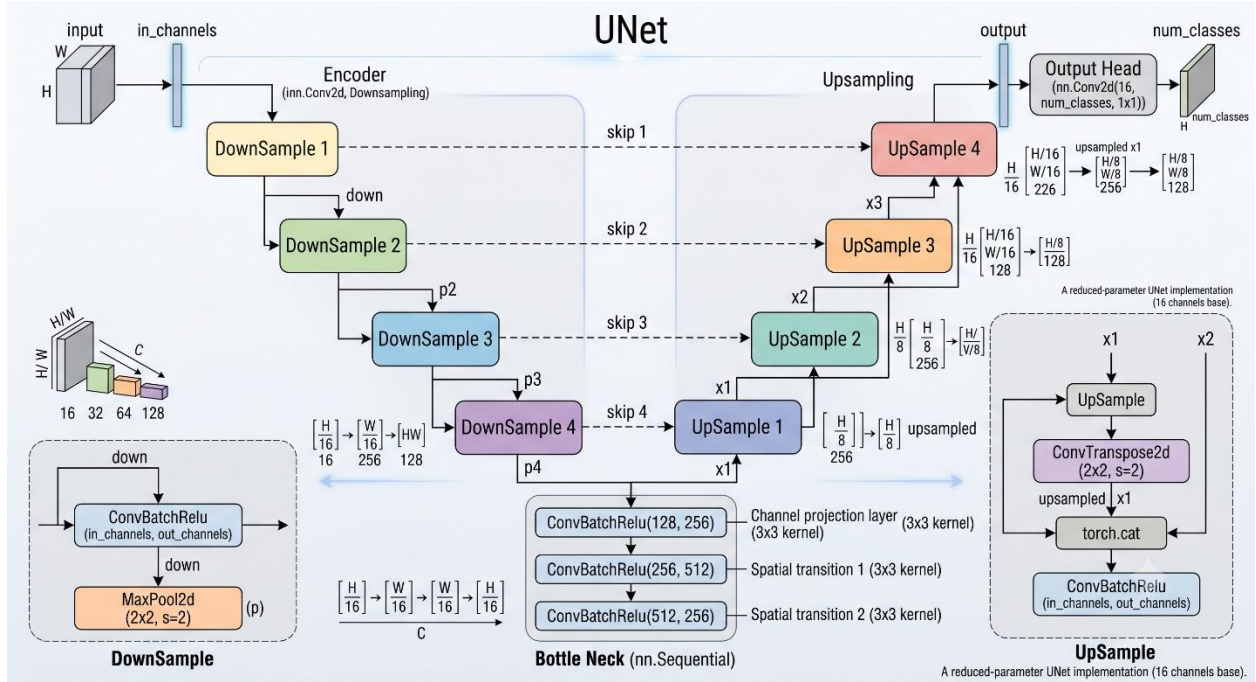


Figure 3 Modified UNet (image generated with assistance of Gemini AI Model based on authors UNet code)

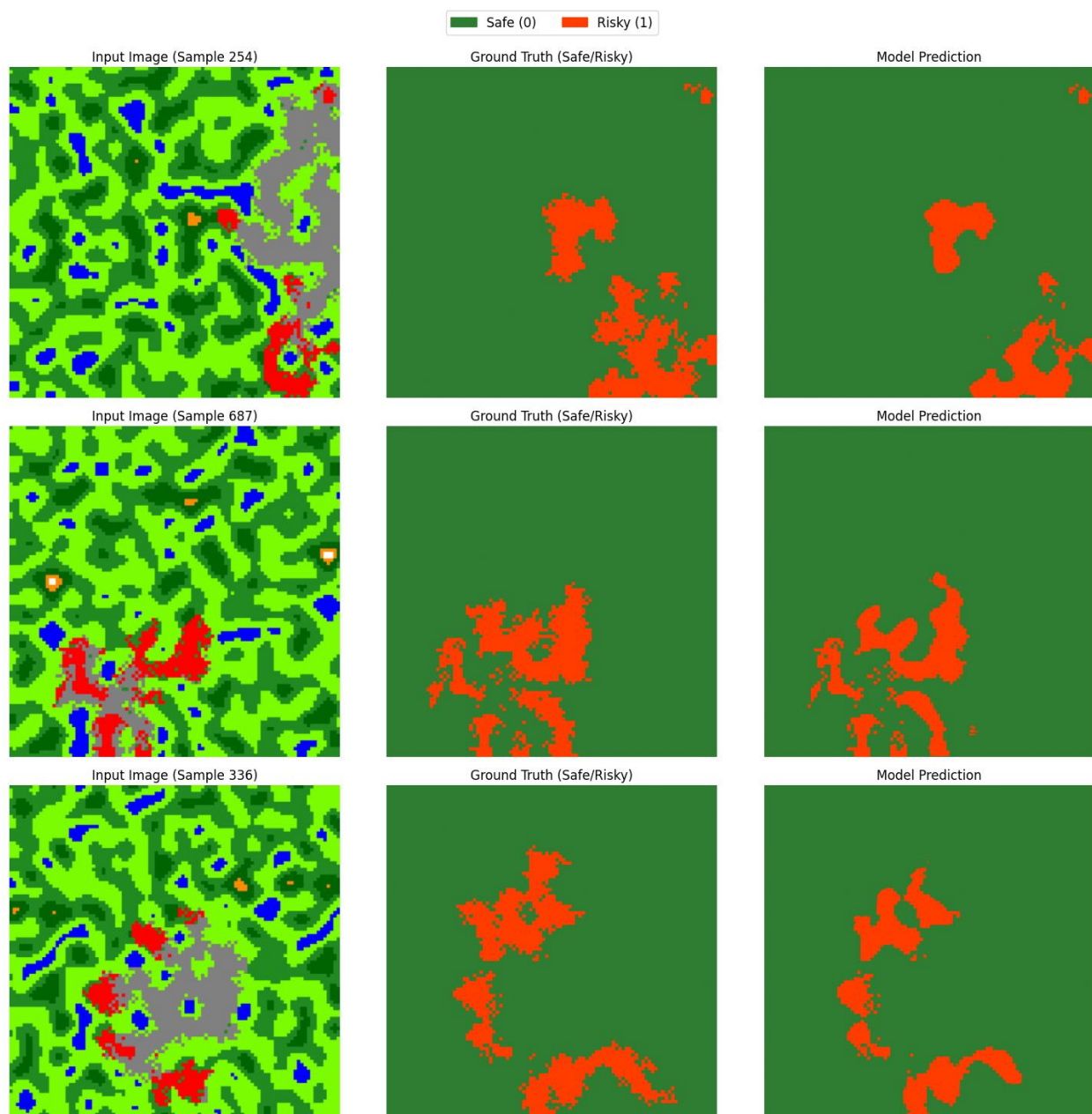


Figure 4 Fire Risk Prediction Outputs

Semi-Autonomous Wildfire Monitoring & Control System Framework Gene Foxwel 2026

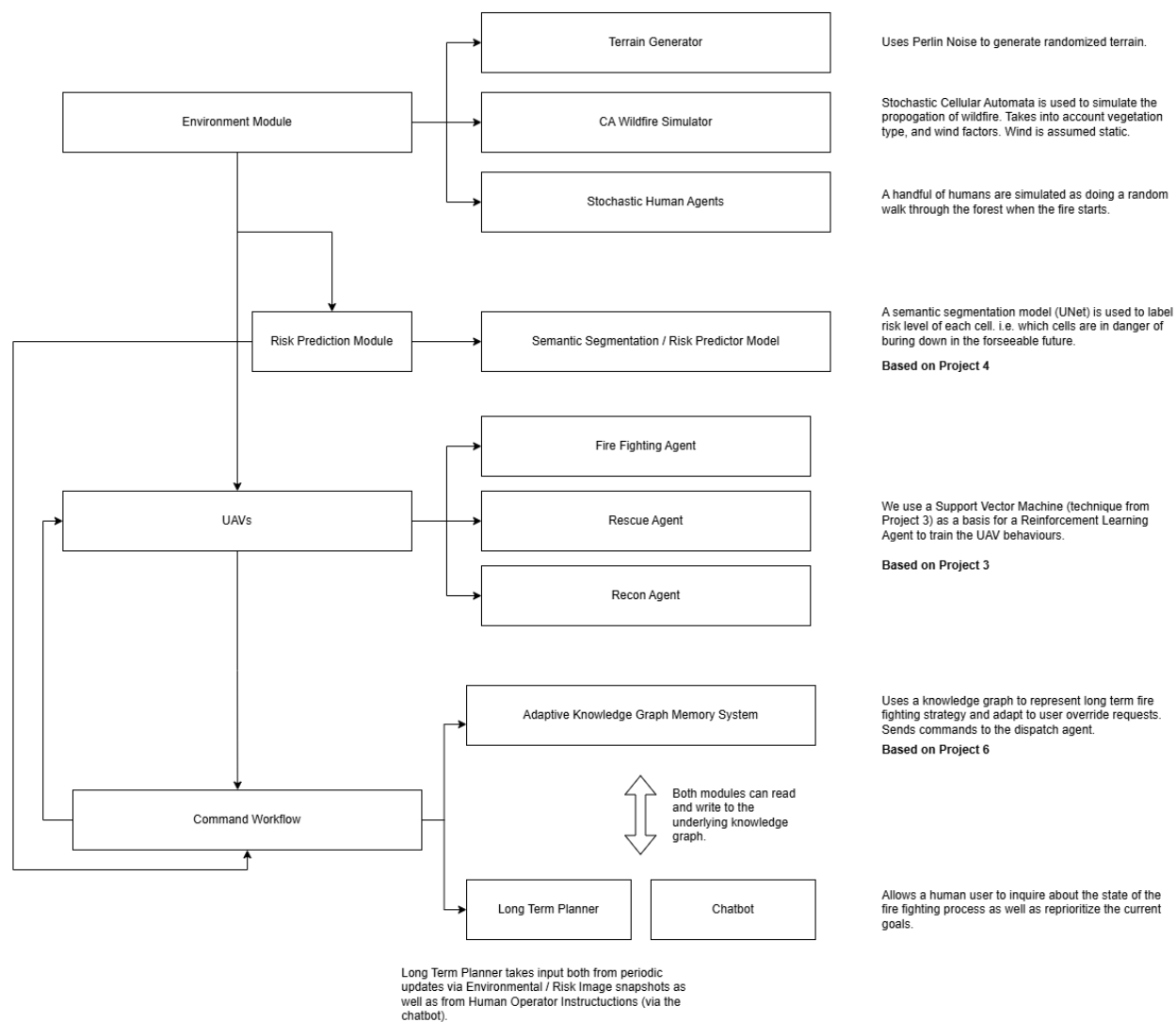
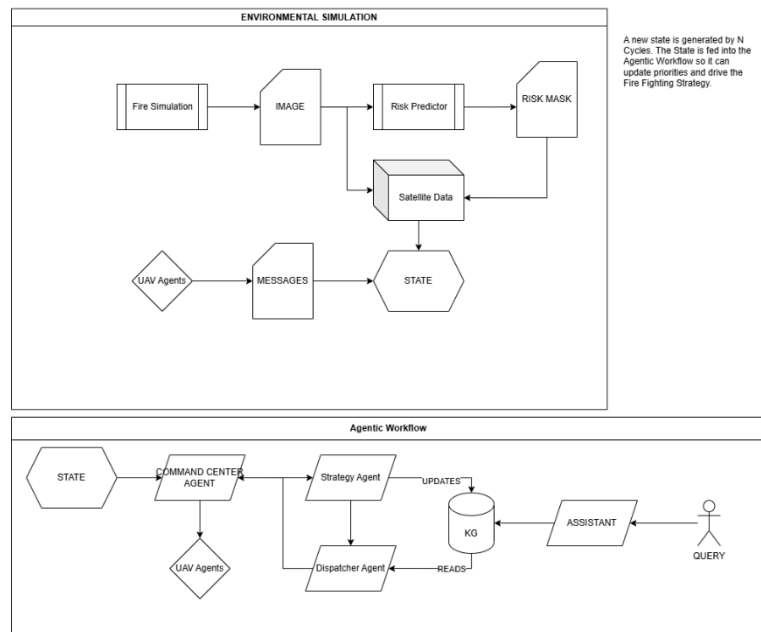


Figure 5 System Overview

Wild Fire Command and Control Workflow



ONTOLOGY

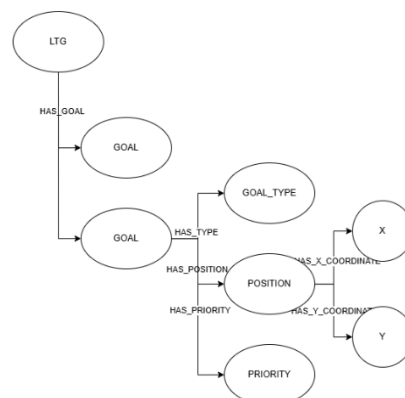


Figure 6 Agentic Workflow

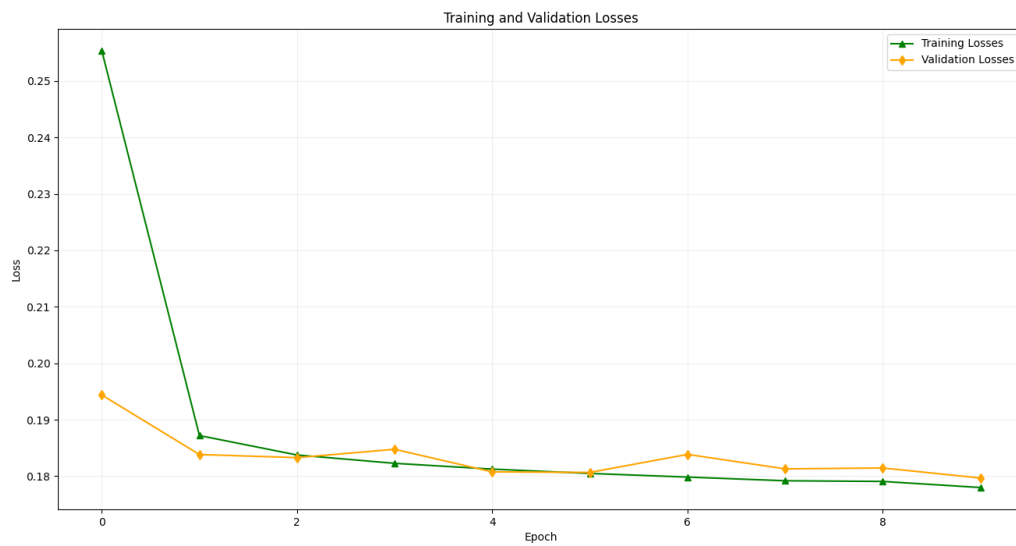


Figure 7 UNet Training and Validation Loss

UAV Behavior Diagram

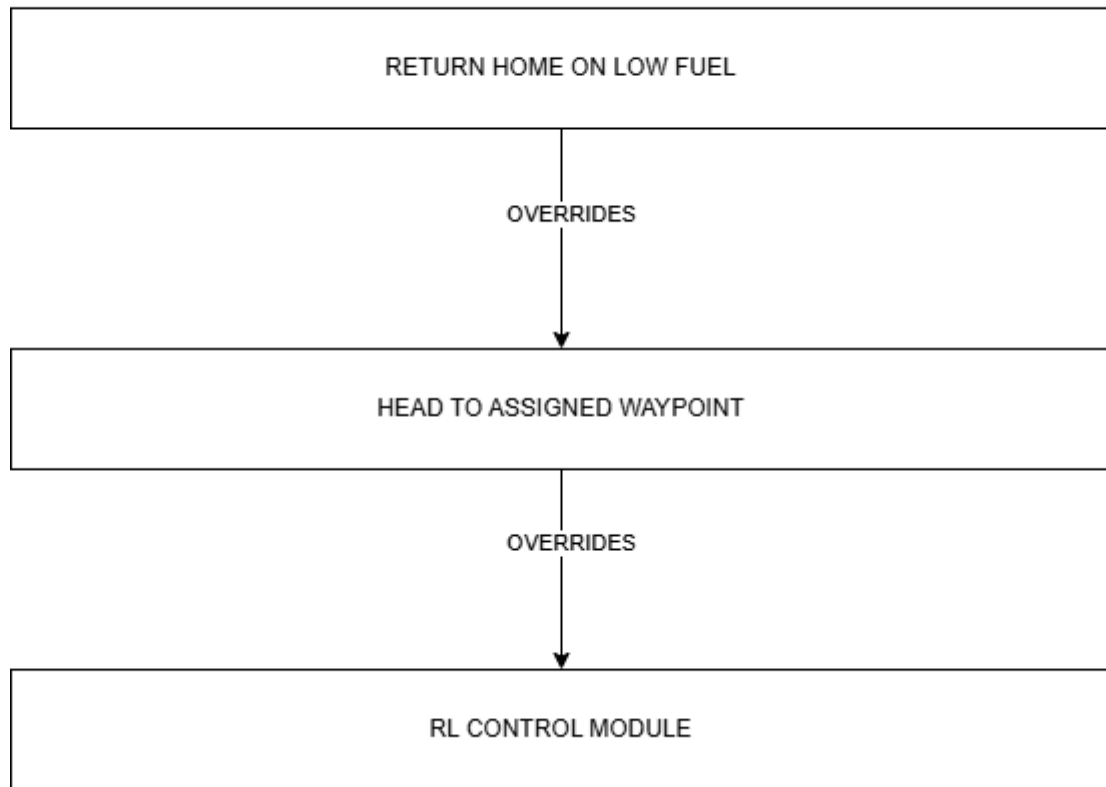


Figure 8 UAV Behavior

Ethical Considerations and Responsible AI

While, as previously pointed out, fighting wildfires is a dangerous occupation, it is still an occupation that provides employment. A fully automated system fighting fires in place of human beings would displace this group, potentially leaving them without a comparable source of income. This is not an intended outcome of this framework. It would be better if any system based on this or similar frameworks were used to extend the reach of already existing human resources – augmenting their fire fighting abilities rather than replacing it.

We also do not recommend extending this system to be fully autonomous. As shown even in the data already collected, Agents (and the LLMs that power them) can sometimes fixate on unexpected things. An Agent that continues to search for a long dead human while ignoring others that may be in danger (as happened a few times during testing) is not a desirable situation.

Many assumptions were made during testing that may not hold up in the real world – or would at least require additional testing. For example, for simplicities sake, we assumed a static wind field. Wind is rarely, if ever, static. We assumed we could obtain reliable aerial views of the fire. We assumed that reliable Firefighting and search and rescue drones could be built reliably. This may not be the case. Furthermore, especially for search and rescue, a human presence is likely preferable to deal with any first-aid or psychological distress that might occur during a potential emergency.

Another concern to consider is who is responsible for loss of life / property damage that may occur due to reliance on a system like this? If it malfunctions or makes a choice between priorities that we might not agree with causing loss of life or property – someone will need to be held accountable. It's for this reason we left in the human-in-the-loop option to override the system, but many people will still defer to the computer, especially when they are stressed. This could result in costly, but potentially avoidable mistakes. Protocols will need to be put in place to review the plans and reasoning presented by the system to ensure they meet the expected standards, even in life and death situations.

Our setup used a remote model – Anthropic Claude Sonnet 4.5. This was sufficient for our purposes, but in a real scenario internet will likely not be reliable

enough to stake human life upon. If a system of this nature were to be deployed, it should use a local model instead – this way if there is a disruption in internet infrastructure – the central command system doesn’t go offline.

Evaluation and Reflection

We ran five end to end tests on the current version of the system. (More tests were run on previous iterations to fine-tune the system). Despite the stochastic nature of the system, we kept the number of tests low as running the system with an appropriate model was cost prohibitive.

For each test we tracked the number of steps taken (up to a maximum of 750), number of humans still active in the forest, and number of humans rescued. We also tracked the final number of burning tiles once the simulation was completed. These results are summarized in the table below.

We used Claude Sonnet 4.5 as our LLM for all Agents.

	STEPS	ALIVE HUMANS	RESCUED HUMANS	BURNING CELLS
EX 1	750	7	3	375
EX 2	72	13	2	0
EX 3	750	2	0	1579
EX 4	750	1	2	1590
EX 5	750	7	0	1585

We notice that in experiment 2 the system obtained significantly better performance than the others. In this experiment we had a bit of a lucky start in that the fire ignited near enough to home base that the Extinguishers were able to reach and put out the fire before it could spread.

Each experiment also resulted in a video file. After reviewing the video files, we noticed the following behavior patterns:

Firefighting UAVs can efficiently fight the fire in their assigned area but will ignore any fire nearby. This could be an artifact of their reward function which punishes them from going too far away from their last assigned waypoint. This results in fire that sometimes spreads further than might otherwise be expected.

Recon UAVs appear to be deployed in clumps near the fire's frontier. It might be better to encourage the system to spread them out a bit to make it easier to track the location of humans in need of rescue.

Search and Rescue operations seem to get fixated on saving a particular human sometimes. Its possible this is caused by the fact the system has no way of differentiating individual humans from each other – so if the one its searching for dies, it doesn't know that and keeps looking in the last known location.

The system quickly gets overwhelmed once the fire passes a certain size. This is particularly noticeable when all the UAVs return to the base for refueling. This might be a defect in the Dispatchers prompt, which instructs the system to always provide a UAV command. As such, it ignores suggestions from the Strategy Agent to hold UAVs in reserve to handle just such an occasion.

In addition, we also kept a log of all commands given to the UAVs during each simulation. We fed these to an LLM to look for any patterns that might be observable purely from the reasons provided by the Command Center Agent.

RESCUE UAV target stickiness: In ex3 and ex5 (both 0 rescues), the RESCUE UAV was locked onto the same coordinates for 8–10 consecutive dispatch cycles (160–200 sim steps). The knowledge graph records "rescue humans at (x,y)" once and nobody ever marks it complete, so the dispatcher keeps sending UAV 7 to the same stale location.

Out-of-bounds coordinates: In ex1, pixel (220, 760) got converted to grid (44, 152) — Y=152 is way off the 100×100 grid. The UAV hits boundary repulsion and never arrives.

Speculative RESCUE deployment: The RESCUE UAV is frequently sent "just in case" to housing areas with language like "once RECON identifies civilians" rather than being dispatched reactively to a confirmed human sighting.

Knowledge graph goals never expire: This is the root cause of the stickiness pattern — stale goals persist indefinitely in the graph.

RECON and EXTINGUISH are generally well-behaved — good role adherence and sensible targeting, though EXTINGUISH sometimes gets inconsistent pixel→grid conversions from the LLM guessing at coordinates.

Based on the above, one major problem appears to be that the system never let's go of a goal once it's been entered into the graph. Two possible improvements come to mind that could address this:

1. Allow Goal nodes to expire if not deleted after a certain number of iterations.
2. Allow for a hybrid prioritization model where the priority of goal is a combination of the original priority and how long it's been active for. So newer Critical priority goals can take precedence over older ones.

Professional Relevance

This project demonstrates several techniques in Artificial Intelligence that may be of interest in a professional setting.

We demonstrate using Semantic Segmentation as a predictive technique – which could have potential applications in Autonomous Driving (Feng 2021) and Robotics (Hurtado 2022) (as well as the original application of UNet to medical imagery).

We demonstrate the use of Reinforcement Learning (RL) as well as Machine Learning via an SVM. RL has already been applied to UAV control in this domain (Julian 2018). RL has also shown to have applications to robotics (Rajeswaran 2017) and automated gameplaying (Mnih 2015).

We also demonstrate the use of Agentic Workflows, which is finding applications in Finance (Yang 2024), and Entertainment (Ammanabrolu 2020) (amongst other things).

We demonstrated that these techniques could be combined to create something potentially useful – including using a simulation environment informed by modern mathematical models of wildfire propagation. Providing a framework for testing system hypotheses without

having to spend the time, money, and risk involved in building a real-world system is an important step in any embodied AI Project.

References

Ammanabrolu, P., Cheung, W., Tu, D., Broniec, W., & Riedl, M. O. (2020). Bringing stories alive: Generating interactive fiction worlds. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*.

<https://arxiv.org/pdf/2001.10161>

Belval, E. J., Pietruszka, B. M., & Viktora, A. (2025). The price of doing business: Severe injuries in wildland firefighters in the United States by activity performed and hazard encountered. *International Journal of Wildland Fire*.

<https://research.fs.usda.gov/treesearch/69384>

Centers for Disease Control and Prevention. (2014). Aviation-related wildland firefighter fatalities — United States, 2000–2013. *Morbidity and Mortality Weekly Report*.

<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4584834/>

Chen, L., Lu, K., Rajeswaran, A., Lee, K., Grover, A., Laskin, M., Abbeel, P., Srinivas, A., & Mordatch, I. (2021). *Decision transformer: Reinforcement learning via sequence modeling*. arXiv. <https://doi.org/10.48550/arXiv.2106.01345>

Connell, J. H. (1990). *Minimalist mobile robotics: A colony-style architecture for an artificial creature*. Academic Press.

Feng, D., Haase-Schutz, C., Rosenbaum, L., Hertlein, H., Glaser, C., Timm, Fabian., Wiesbeck, W., & Dietmayer, K. (2021). Deep Multi-Modal Object Detection and Semantic Segmentation for Autonomous Driving: Datasets, Methods, and Challenges. *IEEE Transactions on Intelligent Transportation Systems*, 22(3), 1341-1360.

<https://doi.org/10.1109/tits.2020.2972974>

Ghosh, R., Adhikary, J., & Chemlal, R. (2024). *Fire spread modeling using probabilistic cellular automata*. arXiv. <https://doi.org/10.48550/arXiv.2403.08817>

Han, H., Ma, L., Shomer, H., Wang, Y., Lei, Y., Guo, K., Hua, Z., Long, B., Liu, H., Aggarwal, C. C., & Tang, J. (2025). *RAG vs. GraphRAG: A systematic evaluation and key insights*. arXiv. <https://doi.org/10.48550/arXiv.2502.11371>

Hurtado, J. V., & Valada, A. (2022). Semantic scene segmentation for robotics. *Deep Learning for Robot Perception and Cognition*, 279-311. Elsevier. <https://doi.org/10.1016/b978-0-32-385787-1.00017-8>

Julian, K. D., & Kochenderfer, M. J. (2018). *Distributed wildfire surveillance with autonomous aircraft using deep reinforcement learning*. arXiv. <https://doi.org/10.48550/arXiv.1810.04244>

Kamble, A. D., & Ghule, S. D. (2025). Procedural terrain generation using Perlin noise. *International Journal of Innovative Research in Technology*, 12(1), 256–259.

https://ijirt.org/publishedpaper/IJIRT180167_PAPER.pdf

Merlin, P. W. (2009). *Ikhana Unmanned Aircraft System: Western States fire missions* (NASA SP-2009-4544; Monographs in Aerospace History, No. 44). National Aeronautics and Space Administration. <https://www.nasa.gov/wp-content/uploads/2023/03/sp-4544.pdf>

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533. <https://doi.org/10.1038/nature14236>

Rajeswaran, Aravind, et al. ‘Learning Complex Dexterous Manipulation with Deep Reinforcement Learning and Demonstrations’. CoRR, vol. abs/1709.10087, 2017, <http://arxiv.org/abs/1709.10087>

Ramadan, A. (2024). *Wildfire autonomous response and prediction using cellular automata (WARP-CA)*. arXiv. <https://doi.org/10.48550/arXiv.2407.02613>

Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. *arXiv*. <https://arxiv.org/abs/1505.04597>

Wang, W., Wang, X., Flannigan, M. D., et al. (2025). Canadian forests are more conducive to high-severity fires in recent decades. *Science*, 387, 91–97.
<https://doi.org/10.1126/science.adol006>

Watkins, C.J.C.H., Dayan, P. *Q*-learning. *Mach Learn* 8, 279–292 (1992).
<https://doi.org/10.1007/BF00992698>

Yang, H., Chen, J., Siew, M., Lorigo-Botran, T., & Joe-Wong, C. (2025). *LLM-powered decentralized generative agents with adaptive hierarchical knowledge graph for cooperative planning*. *arXiv*. <https://doi.org/10.48550/arXiv.2502.05453>

Yang, H., Zhang, B., Wang, N., Guo, C., Zhang, X., Lin, L., Wang, J., Zhou, T., Guan, M., Zhang, R., & Wang, C. D. (2024). *FinRobot: An Open-Source AI Agent Platform for Financial Applications using Large Language Models*. *arXiv*.
<https://doi.org/10.48550/arXiv.2405.14767>

Yu, R., Wan, S., Wang, Y., Gao, C. X., Gan, L., Zhang, Z., & Zhan, D. C. (2025). *Reward models in deep reinforcement learning: A survey*. *arXiv*.
<https://doi.org/10.48550/arXiv.2506.15421>